

AT10 - Lista de Exercícios: For e Percurso de Strings

Desenvolva programas em Python para resolver os problemas abaixo.

Orientações desta lista:

- Resolva primeiro **no papel**, organizando entrada, processamento e saída antes de codificar
- Pratique a **escrita de código no papel**, com atenção à indentação e ao controle das repetições
- Utilize **input()**, **print()**, variáveis, operadores aritméticos, relacionais e lógicos
- Utilize a estrutura de repetição **for**, com **range()** quando necessário, e também o percurso de strings

Em cada exercício, tente identificar com clareza:

quantas repetições serão feitas → o que muda a cada volta → o que deve ser acumulado, contado ou analisado → qual resultado deve ser exibido ao final

Exercício 1 Tabuada completa

Escreva um programa que leia um número inteiro **n** e mostre sua **tabuada de 1 até 10**.

Para cada linha, mostre a multiplicação e o resultado correspondente.

Saída esperada:

```
Digite um número: 7
7 x 1 = 7
7 x 2 = 14
7 x 3 = 21
...
7 x 10 = 70
```

Exercício 2 Fatorial com for

Escreva um programa que leia um número inteiro positivo **n** e calcule o seu **fatorial** usando **for**.

Lembre-se de que:

$$n! = n \times (n-1) \times (n-2) \times \dots \times 1$$

Exemplo:

```
5! = 5 × 4 × 3 × 2 × 1 = 120
```

Saída esperada:

```
Digite um número: 5  
Fatorial: 120
```

Exercício 3 Sequência de Fibonacci

Escreva um programa que leia um número inteiro positivo **n** e imprima os **n primeiros termos** da sequência de Fibonacci.

Nessa sequência, cada termo é calculado a partir da soma dos **dois termos anteriores**.

Ela começa assim:

```
0, 1, 1, 2, 3, 5, 8, 13, ...
```

Saída esperada:

```
Digite a quantidade de termos: 7  
0  
1  
1  
2  
3  
5  
8
```

Exercício 4 Soma dos divisores de um número

Escreva um programa que leia um número inteiro positivo **n** e calcule a **soma de todos os divisores** desse número.

Um divisor de **n** é um número inteiro que divide **n** sem deixar resto.

Exemplo: os divisores de 6 são 1, 2, 3 e 6.

Saída esperada:

```
Digite um número: 6  
Soma dos divisores: 12
```

Exercício 5 Verificação de número primo

Escreva um programa que leia um número inteiro positivo **n** e verifique se ele é **primo**.

Um número primo é aquele que possui **exatamente dois divisores**: 1 e ele mesmo.

Saída esperada:

```
Digite um número: 13  
O número é primo
```

```
Digite um número: 12  
O número não é primo
```

Exercício 6 Palavra invertida

Escreva um programa que leia uma palavra e construa uma nova string com as letras em **ordem inversa**.

Ao final, exiba a palavra invertida.

Saída esperada:

```
Digite uma palavra: python  
Palavra invertida: nohtyp
```

Exercício 7 Verificação de palíndromo

Escreva um programa que leia uma palavra e verifique se ela é um **palíndromo**, ou seja, se pode ser lida da mesma forma da esquerda para a direita e da direita para a esquerda.

Exemplos de palíndromos: **arara**, **ovo**, **radar**.

Saída esperada:

```
Digite uma palavra: arara  
A palavra é um palíndromo
```

Exercício 8 Soma dos dígitos de um número

Escreva um programa que leia um número inteiro positivo e calcule a **soma de seus dígitos**.

Neste exercício, trate o número digitado como uma **string** para percorrer seus caracteres com **for**.

Exemplo: no número 2537, a soma dos dígitos é $2 + 5 + 3 + 7 = 17$.

Saída esperada:

```
Digite um número: 2537
Soma dos dígitos: 17
```

Exercício 9 Número perfeito

Escreva um programa que leia um número inteiro positivo **n** e verifique se ele é um **número perfeito**.

Um número é perfeito quando ele é igual à soma de seus **divisores positivos menores que ele mesmo**.

Exemplo: 6 é perfeito, pois $1 + 2 + 3 = 6$.

Saída esperada:

```
Digite um número: 6
O número é perfeito
```

```
Digite um número: 10
O número não é perfeito
```

Exercício 10 Primos em um intervalo

Escreva um programa que leia dois números inteiros positivos **início** e **fim** e mostre todos os **números primos** existentes nesse intervalo, incluindo os extremos quando for o caso.

Este exercício exige o uso de **repetições aninhadas**: uma para percorrer o intervalo e outra para verificar se cada número é primo.

Saída esperada:

```
Digite o início do intervalo: 10
Digite o fim do intervalo: 20
Números primos no intervalo:
11
13
17
19
```